

Secure Core Banking System (SCBS)

Deliverable 3 - Final Security Report

Security Testing Analysis | Secure Code Review | Final Security Fixes | Final Demo

Course	Secure Software Development and Development (CY321)
Deliverable	Deliverable 3
Project Type	Secure Flask-based core banking web application
Repository Branch	main - final security evidence; original implementation branch referenced frontend
Updated Test Evidence Date	05 May 2026
Team Members	Ali Ahmed Chaudhry 2023093, Muhammad Ismail 2023452, Shayan Siddiqui 2023656
Repository	https://github.com/Ismail-005/Secure-Software-Design-Project

1. Executive Summary

Secure Core Banking System (SCBS) is a role-based banking dashboard implemented with Python, Flask, SQLite, SQLAlchemy, Flask-WTF, Flask-Login, Flask-Session, and Flask-Migrate. The application focuses on controlled money-movement workflows, account management, fraud review, audit visibility, and secure authentication. Deliverable 3 documents the security-focused phase of the project, including threat model coverage, secure testing, secure code review, security fixes, and final demo readiness.

This updated version adds the final security and quality test evidence captured on 05 May 2026. The latest run reports 85 passed tests, 0 failed tests, 92% overall coverage, 0 Bandit security issues, 0 known dependency vulnerabilities from pip-audit, Radon average complexity grade A, and a Pylint score of 8.21/10. These results supersede the earlier 59-passed-test snapshot in the original report.

The implementation contains layered boundaries: routes handle user-facing request validation, services enforce business and security rules, repositories isolate database access, and middleware centralizes access control, rate limiting, and session protection.

2. Deliverable 3 Scope Mapping

Deliverable 3 Requirement	How SCBS Satisfies It
Security Testing Analysis	Security tests cover brute-force lockout, rate limiting, unauthorized access rejection, admin-only audit visibility, replay attack rejection, SQL injection payload handling, XSS escaping, audit tamper detection, and insufficient-funds protection.
Secure Testing Techniques and Analysis	The suite combines integration, security, and unit tests. Integration tests validate Flask routes through the test client; unit tests validate services, repositories, middleware, fraud logic, audit integrity, and transaction rules.
Secure Code Review	Review focuses on authentication, MFA/OTP, session handling, RBAC, CSRF, input validation, database access, financial transaction integrity, audit log integrity, dependency risk, static analysis, and configuration hardening.
Final Security Fixes and Enhancements	Implemented fixes include secrets-based randomness, dependency upgrades for CVEs, timezone-aware datetime handling, session hardening, replay protection, fraud detection, tamper-evident audit logs, and expanded route coverage.
Final Report	This document consolidates threat model, security features, test strategy, latest test results, limitations, final fixes, and demo plan.
Source Code Submission	Source code is maintained in the GitHub repository. Final security evidence was collected on branch main.
Live Demo and Presentation	Demo path covers login, MFA, dashboard, transactions, fraud review, audit logs, admin controls, security rejection cases, and final test evidence.

3. System Overview

SCBS simulates a secure internal banking platform. The core workflows are customer sign-up, staff and customer authentication, OTP verification for coursework MFA, account dashboard access, deposit, withdrawal, transfer, transaction history, fraud review, audit log review, and admin user management.

- Backend: Python 3.11 and Flask.
- Database layer: SQLite with SQLAlchemy and migrations through Flask-Migrate.
- Security extensions: Flask-WTF, Flask-Login, Flask-Session, bcrypt, pytest, pytest-flask, pytest-cov, bandit, pip-audit, pylint, and radon.
- Architecture: Routes -> Services -> Repositories -> SQLite.
- Primary roles: customer, teller, manager, and admin.

4. Final Threat Model

4.1 Protected Assets

- User credentials and password hashes.
- Authenticated sessions and OTP verification state.
- Customer identities, account data, and transaction history.
- Ledger entries used to calculate account balances.
- Fraud flags and review queues.
- Audit logs and audit hash-chain integrity values.
- Administrative privileges and role assignments.

4.2 Trust Boundaries

Boundary	Risk	Control
Browser -> Flask routes	Malicious form input, CSRF, XSS payloads, unauthenticated access.	Flask-WTF CSRF, route validation, login requirements, Jinja escaping, RBAC decorators/middleware.
Routes -> Services	Bypassing business rules or role checks.	Security-sensitive operations are centralized in service methods and role-aware middleware.
Services -> Repositories	SQL injection or inconsistent persistence.	Repository pattern and SQLAlchemy parameterized access.
Transaction request -> Ledger	Replay, insufficient funds, partial updates, incorrect balance.	Nonce-based replay checks, insufficient-funds checks, atomic transaction logic, ledger-backed balances.
Security events -> Audit logs	Silent tampering or missing accountability.	HMAC-backed hash-chained audit logs for tamper evidence.
Configuration -> Runtime app	Weak secrets, unsafe defaults, debug exposure.	Startup rejection for missing/placeholder SECRET_KEY, debug not forced on, secure cookie attributes.

4.3 STRIDE-Based Threat Analysis

STRIDE Category	Representative Threat	SCBS Mitigation
Spoofing	Attacker tries to log in as another user.	bcrypt password verification, OTP verification step, failed-login tracking, account lockout.
Tampering	Attacker modifies transaction or audit data.	Atomic transaction flow, ledger-backed balance calculation, HMAC hash-chain audit logs.
Repudiation	User denies transaction or admin action.	Audit logs record login, transaction, lockout, and security events.
Information Disclosure	Customer accesses another user account or admin-only pages.	Role-based access control and ownership checks.
Denial of Service	Repeated login or transaction attempts overload routes.	Rate limiting and lockout for abusive login behavior.
Elevation of Privilege	Customer attempts to access admin functions.	RBAC restrictions and route-level authorization checks.

5. Implemented Security Features

Control Area	Implementation Summary
Authentication	Username/password login with bcrypt password hashing and verification.
MFA / OTP	Coursework demo OTP verification flow after password authentication. OTP expiry is configurable. OTP generation now avoids insecure random usage.
Brute-force defense	Failed login tracking and account lockout after repeated failures.
Rate limiting	Configurable request throttling window and maximum attempt limits.
Session protection	Session timeout, IP-bound session guard, HTTP-only cookies, SameSite=Lax cookies.
CSRF protection	Flask-WTF CSRF enabled in the default app configuration.
Authorization	RBAC across customer, teller, manager, and admin workflows.
Database safety	SQLAlchemy-backed repository access and parameterized query behavior.
XSS prevention	Jinja template auto-escaping and security tests for XSS payload escaping.
Replay prevention	Transaction nonces prevent duplicate replay submission.
Financial integrity	Ledger-backed balance calculation, insufficient-funds protection, and atomic transaction processing.
Fraud detection	Large transfer, new account transfer, and high-velocity transaction checks.
Audit integrity	HMAC-backed tamper-evident audit log hash chain.
Configuration hardening	Application rejects missing or placeholder production SECRET_KEY values.
Dependency hygiene	pip-audit was run after upgrades and reports no known vulnerabilities.

6. Secure Testing Strategy

The project uses a mixed testing strategy. Integration tests exercise full Flask route behavior through the Flask test client. Security tests simulate adversarial web and banking cases. Unit tests validate the correctness of services, repositories, middleware, fraud rules, audit chain generation, and transaction service behavior. The final evidence also includes tool-based static security analysis, dependency audit, complexity analysis, style analysis, and coverage reporting.

Test Layer	Purpose	Examples
Integration	Validate end-to-end route behavior and authentication flow.	Login page, sign-in alias, signup, MFA page, logout, dashboard login requirements, account dashboard, deposit route, insufficient-funds withdrawal.
Security	Validate resistance to common web and banking attacks.	Brute-force lockout, rate limiting, unauthorized admin access rejection, replay rejection, SQL injection handling, XSS escaping.
Unit	Validate isolated logic and internal security controls.	Password hashing, OTP verification, account creation, balance calculation, audit hash-chain verification, fraud detection, RBAC, rate limiter, transaction service.
Static and quality tools	Validate codebase risk indicators outside runtime tests.	Bandit, pip-audit, pylint, radon, pytest-cov.

7. Security Test Analysis

Security Scenario	Technique	Expected Secure Behavior	Reported Result
Brute-force login	Repeated failed login attempts.	Account is locked after configured failed-attempt threshold.	Covered by security tests.
Rate limit abuse	Repeated requests inside configured time window.	Requests beyond the limit are blocked/throttled.	Covered by security tests.
Unauthenticated dashboard access	Direct request to protected dashboard URL.	User is redirected or rejected until authenticated.	Covered by integration/security tests.
Customer attempts admin access	Authenticated customer requests admin route.	Access is denied because customer role lacks admin privileges.	Covered by security tests.
Audit visibility	Admin accesses audit log interface.	Authorized admin can view audit logs; non-authorized roles cannot.	Covered by security tests.
Replay attack	Duplicate transaction submission with reused nonce.	Repeated transaction is rejected.	Covered by security and unit tests.
SQL injection	Authentication/input payload contains SQL injection patterns.	Payload is handled as data, not executable SQL.	Covered by security tests.
XSS	Script payload submitted into rendered fields.	Payload is escaped in template output.	Covered by security tests.
Audit log tampering	Audit chain verification is run after log creation.	Chain verification detects inconsistency if records are changed.	Covered by unit tests.
Insufficient funds	Withdrawal/transfer exceeds calculated balance.	Transaction fails and balance is not corrupted.	Covered by integration/unit tests.

8. Secure Code Review

8.1 Authentication and Credential Handling

The code review confirms that plaintext password storage is avoided by using bcrypt hashing. Authentication is reinforced by failed-login tracking and lockout behavior. OTP and account-number randomness were changed from Python random to the secrets module, removing the CWE-330 weak-randomness finding identified during earlier testing.

8.2 Authorization and Access Control

The authorization model distinguishes customer, teller, manager, and admin responsibilities. Customers must not access institution-wide account data, fraud review, audit logs, or user-management functions. Tests explicitly cover customer restriction on admin pages and admin access to audit logs.

8.3 Input Handling and Web Attack Resistance

The application relies on Flask-WTF for CSRF protection in the default configuration. Jinja2 template escaping is used for frontend rendering, and security tests exercise XSS payload escaping. SQLAlchemy-backed repository access reduces SQL injection risk by preventing string-concatenated SQL execution patterns in normal repository usage.

8.4 Transaction Integrity

The banking domain makes transaction integrity more important than visual completeness. SCBS uses ledger-backed balance calculation instead of trusting a manually stored balance field. Deposit, withdrawal, and transfer behavior is tested, including insufficient-funds rejection and replay prevention through nonces.

8.5 Auditability and Tamper Evidence

Audit logging is treated as a security control. The system records login, transaction, account lock, and other security events. HMAC-backed hash chaining provides tamper evidence: changing prior audit records should invalidate later chain verification.

8.6 Configuration Review

The configuration rejects missing or placeholder production SECRET_KEY values at application startup, while the testing configuration uses a separate test secret and test database. Cookie settings include HTTP-only behavior and SameSite=Lax. Flask debug mode is not forced on by the application factory.

9. Final Security Fixes and Enhancements

Fix / Enhancement	Security Value
Placeholder SECRET_KEY rejection	Prevents the app from starting in non-test mode with known weak placeholder secrets.
CSRF-enabled default configuration	Protects state-changing form submissions against cross-site request forgery.
IP-bound session guard	Raises the cost of session hijacking by tying sessions to the client IP context.
Session timeout behavior	Limits exposure from abandoned or stale authenticated sessions.
Rate limiting and login lockout	Reduces brute-force and credential stuffing success probability.
Transaction nonce replay checks	Stops duplicate transaction submissions from being accepted.
Fraud detection workflow	Flags high-risk money movement for manager/admin review.
Tamper-evident audit logs	Improves accountability and makes silent audit manipulation detectable.
Weak randomness removal	Replaced security-sensitive random usage with the secrets module for OTP and account-number generation.
Dependency upgrades	Resolved three identified CVEs by upgrading Flask, python-dotenv, and pytest.
Datetime modernization	Replaced datetime.utcnow() with timezone-aware datetime.now(timezone.utc) and normalized DB-stored datetimes.
Expanded test coverage	Added admin and transaction route integration tests, raising overall coverage from 83% to 92%.
Frontend redesign with role-aware surfaces	Makes fraud, audit, admin, and transaction workflows visible and demo-ready.

10. Updated Security and Quality Test Results

The final test evidence below supersedes the earlier snapshot that reported 59 passed tests. The updated run used bandit, pylint, radon, pip-audit, and pytest-cov on branch main.

Tool / Metric	Updated Result	Interpretation
pytest / pytest-cov	85 passed, 0 failed; 92% overall coverage	Passes the 80% coverage threshold and substantially improves over the previous 83% baseline.
Bandit	0 issues; 1,036 lines scanned	No low, medium, or high static security findings remain.
pip-audit	No known vulnerabilities found	Previously identified CVEs were resolved through dependency upgrades.
Pylint	8.21 / 10	Acceptable quality score. Remaining warnings are convention/style only, not functional or security issues.
Radon	Average complexity A (1.82)	No grade C or worse blocks. Highest-risk functions are still B-level only.

10.1 Bandit - Security Static Analysis

Bandit reported 0 issues after scanning 1,036 lines. Low, medium, and high issue counts were all 0. The earlier weak-randomness issue was addressed by replacing random usage with the secrets module, removing CWE-330 exposure.

```
No issues identified.
Total lines scanned: 1,036
Total issues (Low): 0
Total issues (Medium): 0
Total issues (High): 0
```

10.2 Dependency Audit - pip-audit

pip-audit reported no known vulnerabilities after dependency upgrades. The resolved items are shown below.

Package	Old Version	CVE	Fixed Version
flask	3.0.3	CVE-2026-27205	3.1.3
python-dotenv	1.0.1	CVE-2026-28684	1.2.2
pytest	8.2.0	CVE-2025-71176	9.0.3

10.3 Coverage - Test Results

The updated coverage report shows 85/85 tests passed and 92% overall coverage. This is a material improvement over the earlier report snapshot. Admin routes improved from 58% to 99%, transaction routes improved from 74% to 93%, all repository modules reached 100%, and transaction_service improved from 79% to 98%.

Module	Statements	Missed	Coverage
app/__init__.py	28	1	96%
app/config.py	38	2	95%
app/extensions.py	10	0	100%
app/middleware/rate_limiter.py	20	1	95%
app/middleware/rbac.py	14	1	93%
app/middleware/session_guard.py	26	6	77%
app/models/*	103	0	100%
app/repositories/*	105	0	100%
app/routes/accounts.py	29	2	93%
app/routes/admin.py	111	1	99%
app/routes/auth.py	95	20	79%
app/routes/transactions.py	92	6	93%
app/seed.py	20	20	0% - seeding script
app/services/account_service.py	22	2	91%
app/services/audit_service.py	32	1	97%
app/services/auth_service.py	64	5	92%
app/services/fraud_service.py	24	1	96%
app/services/transaction_service.py	62	1	98%
TOTAL	902	70	92%

10.4 Remaining Coverage Gaps

Module	Coverage	Uncovered Lines / Reason
app/routes/auth.py	79%	MFA/signup edge-case flows: lines 40, 47, 63-64, 72, 83-84, 98, 103-111, 123-125.
app/middleware/session_guard.py	77%	IP-mismatch and timeout paths: lines 12, 17, 20-21, 24-25.
app/seed.py	0%	Seeding script, not unit-testable in the normal app test suite.

10.5 Complexity and Code Quality

Radon reports average complexity grade A with average complexity 1.82. No grade C or worse blocks remain. The highest-complexity functions are still acceptable B-level blocks: transfer has complexity 7; FraudService.evaluate, deposit, withdraw, reject_fraud, and AuthService.attempt_login have complexity 6. AuthService.attempt_login was reduced from complexity 9 to 6 after extracting _resolve_lock_status.

Pylint reports 8.21/10, improved from 0.00/10. The remaining warnings are convention-level only: enum member naming aligned to SQLAlchemy DB values, short argument name ip in service methods, and import ordering in selected service/repository files. No functional or security warnings remain.

10.6 Cumulative Fixes Applied

Priority	Fix	Status
HIGH	random -> secrets for OTP and account number generation (CWE-330)	Done
HIGH	Upgraded Flask, python-dotenv, and pytest to resolve three CVEs	Done
MEDIUM	datetime.utcnow() -> datetime.now(timezone.utc) across app and test files	Done
MEDIUM	Naive/aware datetime normalization for DB-stored datetimes	Done
MEDIUM	Added 15 admin route integration tests: 58% -> 99%	Done
MEDIUM	Added 9 transaction route integration tests: 74% -> 93%	Done
LOW	Refactored attempt_login by extracting _resolve_lock_status: complexity 9 -> 6	Done
LOW	Suppressed Pylint R0913 with inline comments on service/repo methods	Done
LOW	Removed unused db import in transaction_service.py	Done
LOW	Fixed import ordering where practical	Done

11. Final Assessment

SCBS is now in a significantly stronger security posture than the earlier Deliverable 3 snapshot. The project no longer has known dependency vulnerabilities from pip-audit, no Bandit findings, no failed tests, and 92% coverage. The most important banking-specific security properties - authorization boundaries, replay resistance, ledger-backed balance calculation, fraud review, insufficient-funds protection, and audit tamper evidence - are represented in the test strategy.

The remaining weaknesses are not ignored: auth.py and session_guard.py remain slightly under 80% coverage, and seed.py is intentionally outside the normal testable app path. These are not fatal for the coursework submission, but they are the correct next targets if the codebase were being hardened for production. The blunt conclusion: the updated evidence is strong enough for Deliverable 3; the original 59-test report snapshot is now outdated and should not be submitted as-is.

12. Final Demo Plan

Demo Step	What to Show	Security Point
1	Login and MFA/OTP flow	Authentication requires password and second verification step.
2	Customer dashboard and account view	Customer role sees only authorized account surfaces.
3	Deposit, withdrawal, and transfer	Ledger-backed balance logic and insufficient-funds rejection.
4	Replay attempt with reused transaction nonce	Duplicate transaction is rejected.
5	Customer attempts admin route	RBAC rejects unauthorized access.
6	Admin audit log view	Privileged audit visibility and accountability.
7	Fraud review queue	High-risk transaction workflow is visible.
8	Run/show final test summary	85/85 tests passed, 92% coverage, 0 Bandit issues, 0 dependency CVEs.

13. Final Summary

Tool	Result
Bandit	0 issues
Pylint	8.21 / 10
Radon	Average A (1.82); no grade C or above
pip-audit	0 known vulnerabilities
Coverage	92%; 85/85 tests passed
Final Security Status	No known security vulnerabilities remain from the tested and audited items. All medium and high recommendations listed in the update were implemented.